CNN for MNIST Digit Classification

CYSE 635 — Lab Exercise 03 Report

Bongiwe Mkwananzi

10/26/2026

CYSE 635

1. Introduction

The objective of this lab was to develop an understanding of adversarial machine learning focusing on the creation and visualization of adversarial examples using the MNIST dataset. The lab builds on Project 1, where a CNN with three convolutional layers and one fully connected layer was trained to classify handwritten digits from MNIST, and that same architecture was retrained here and reached 99.37 percent test accuracy. The lab prompt asked to implement a method to generate adversarial examples, starting with the Fast Gradient Sign Method (FGSM) and then exploring more complex methods like Projected Gradient Descent (PGD) if time permitted, so this report covers both. FGSM changes an image in a single step along the sign of the loss gradient, while PGD takes many smaller steps and keeps the total change within the same limit ε(perturbation amount). Both methods were applied to create adversarial examples from the test portion of the MNIST dataset, and the original model's performance was evaluated on both clean and adversarial data. The comparison covers accuracy at base ε of 0.08, then accuracy across ε values from 0 to 0.3, and visualizations of the adversarial images and the model's feature maps.

2. Model Implementation

2.1 Architecture Overview

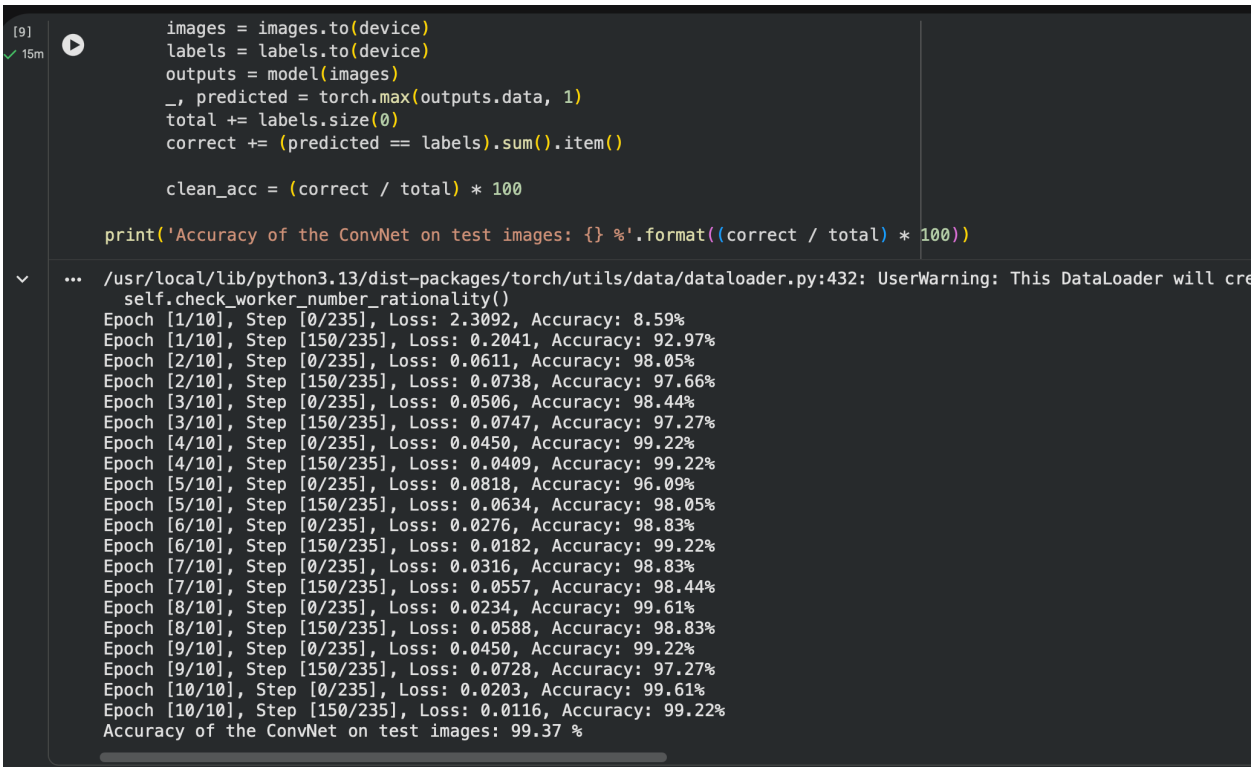
The network is the same model used in Project 1, with three convolutional layers and one fully connected layer, which meets the lab requirement of at least two convolutional layers and one fully connected layer. Each convolutional layer applies a 3 by 3 filter with stride 1 and padding 1, followed by a ReLU activation and a 2 by 2 max pooling layer, and the number of filters widens from 32 to 64 to 128 across the three layers. The classifier flattens the resulting 3 by 3 by 128 volume into 1,152 values, applies dropout at a rate of 0.5 and passes the result to a single fully connected layer with 10 outputs, one for each digit class.

The architecture was kept unchanged from Project 1 so that any drop in accuracy in this lab could be attributed to the adversarial inputs rather than to a change in the model. It also meant the feature map method from that report could be reused directly for the clean versus adversarial comparison in section 5.3.

2.2 Training and clean (no perturbation) baseline

The model was trained from scratch with the Adam optimizer at a learning rate of 0.001 for ten epochs, using cross entropy loss and a batch size of 256 on a GPU runtime [ended up using CPU as I ran out of GPU instances on my current google collab subscription which increased runtime, so training ended up being 15m] , which took about one minute(this is if I continued running with GPU but new runtime on CPU is 15minutes). Images were converted to tensors in the 0 to 1 range with no normalization or augmentation, which matters here because both attacks clip every adversarial image back into that same range.

Evaluated in evaluation mode on all 10,000 test images, the model classified 99.37 percent correctly (see Figure 1 below), which is 9,937 of the 10,000 images and leaves 63 misclassified. This is the clean baseline for every comparison in this report. It differs slightly from the 99.17 percent reported in Project 1 because the network was retrained from a new random initialization.



```
[9]
✓ 15m
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

        clean_acc = (correct / total) * 100

    print('Accuracy of the ConvNet on test images: {} %'.format((correct / total) * 100))

...  /usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:432: UserWarning: This DataLoader will cre
       self.check_worker_number_rationality()
     Epoch [1/10], Step [0/235], Loss: 2.3092, Accuracy: 8.59%
     Epoch [1/10], Step [150/235], Loss: 0.2041, Accuracy: 92.97%
     Epoch [2/10], Step [0/235], Loss: 0.0611, Accuracy: 98.05%
     Epoch [2/10], Step [150/235], Loss: 0.0738, Accuracy: 97.66%
     Epoch [3/10], Step [0/235], Loss: 0.0506, Accuracy: 98.44%
     Epoch [3/10], Step [150/235], Loss: 0.0747, Accuracy: 97.27%
     Epoch [4/10], Step [0/235], Loss: 0.0450, Accuracy: 99.22%
     Epoch [4/10], Step [150/235], Loss: 0.0409, Accuracy: 99.22%
     Epoch [5/10], Step [0/235], Loss: 0.0818, Accuracy: 96.09%
     Epoch [5/10], Step [150/235], Loss: 0.0634, Accuracy: 98.05%
     Epoch [6/10], Step [0/235], Loss: 0.0276, Accuracy: 98.83%
     Epoch [6/10], Step [150/235], Loss: 0.0182, Accuracy: 99.22%
     Epoch [7/10], Step [0/235], Loss: 0.0316, Accuracy: 98.83%
     Epoch [7/10], Step [150/235], Loss: 0.0557, Accuracy: 98.44%
     Epoch [8/10], Step [0/235], Loss: 0.0234, Accuracy: 99.61%
     Epoch [8/10], Step [150/235], Loss: 0.0588, Accuracy: 98.83%
     Epoch [9/10], Step [0/235], Loss: 0.0450, Accuracy: 99.22%
     Epoch [9/10], Step [150/235], Loss: 0.0728, Accuracy: 97.27%
     Epoch [10/10], Step [0/235], Loss: 0.0203, Accuracy: 99.61%
     Epoch [10/10], Step [150/235], Loss: 0.0116, Accuracy: 99.22%
     Accuracy of the ConvNet on test images: 99.37 %
```

Figure 1. Accuracy of ConvNet (clean/ no perturbations)

3. Adversarial Example Generation

NB: For full implementation check the GitHub repository

3.1 Fast Gradient Sign Method (FGSM)

FGSM generates an adversarial image in a single step. It uses the gradient of the loss with respect to the input pixels rather than the weights, which shows the direction each pixel should move to increase the loss. Every pixel is then moved a fixed amount in that direction.

That amount is epsilon (ε), the perturbation magnitude, which is the largest change allowed for any pixel on the 0 to 1 intensity scale. A larger epsilon makes the attack stronger but also more visible. Following the PyTorch definition, the adversarial image is

$$perturbed_image = image + epsilon \cdot sign(data_grad) = x + \varepsilon \cdot sign(\nabla_x J(\theta, x, y))$$

where x is the clean image, y is its true label, θ is the model's weights and J is the loss. The result is clipped to [0, 1] to keep pixel values valid.

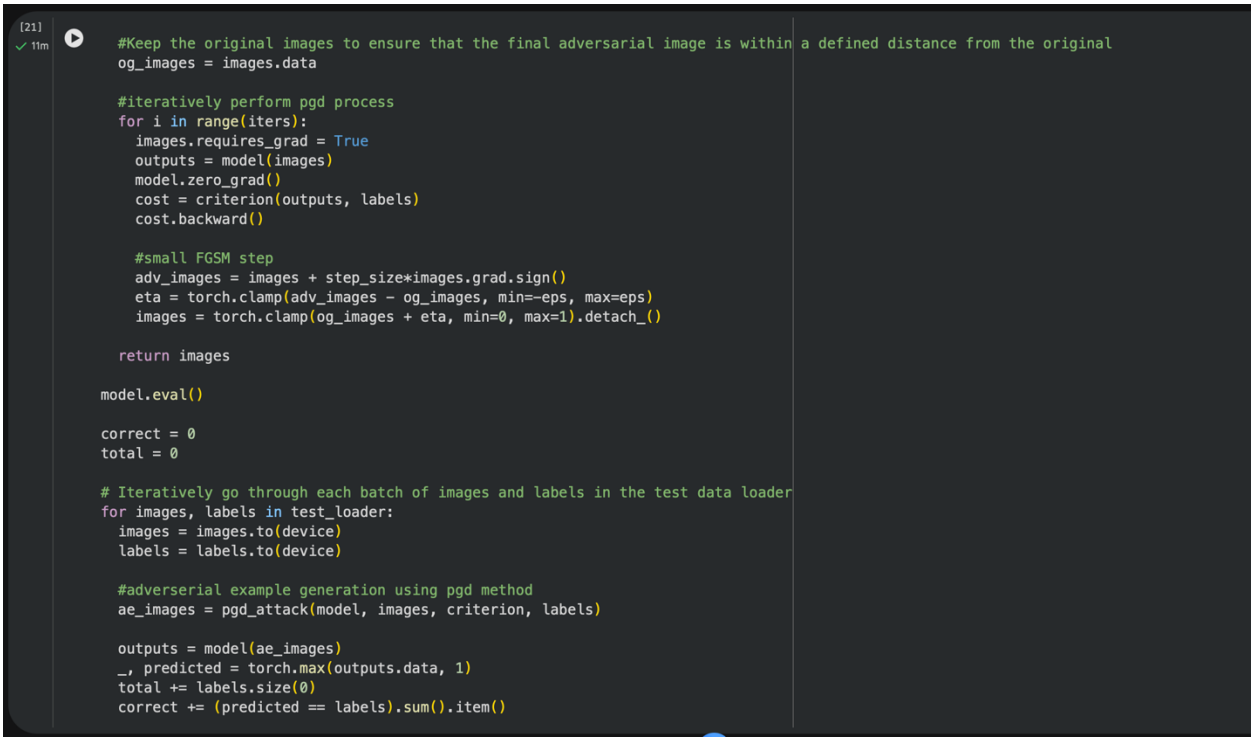
In the fgsm_attack function, gradient tracking is turned on for the images, and the loss is computed against the true labels. Calling backward fills images.grad with $\nabla_x J$. The sign of that gradient is added to the images after scaling by epsilon, and the result is clipped. Taking the sign means every pixel moves by exactly epsilon, so the change is spread evenly across all 784 pixels. FGSM was run at an epsilon of 0.08 for the main comparison and across a range of epsilon values from 0 to 0.3 for the hyperparameter experiment.

3.2 Projected Gradient Descent (PGD)

PGD repeats the FGSM step many times with a much smaller step size. After each step, the total change from the original image is clipped to the range of minus epsilon to plus epsilon, which projects the image back within the allowed perturbation, and the pixels are clipped to the 0 to 1 range again. Since the gradient is recomputed at every

step, PGD can follow the shape of the loss surface. FGSM instead assumes the loss rises in a straight line across the whole range of epsilon.

In the pgd_attack function, the clean images are saved first so every step can be measured against them. Each iteration computes the loss and gradient at the current image, takes a small step in the direction of the gradient sign, then clips the total change and the pixel values before the next iteration. The main comparison used an epsilon of 0.08 with a step size of 0.004 and 40 iterations. Across the range of epsilon values from 0 to 0.3, the step size was set to one tenth of epsilon so that 40 steps could always reach the full epsilon. See Figure 2 below, showing code snippet of a pgd_attack that has small fgsm_attack steps. See Figure 2 below, showing the pgd_attack function, which repeats small FGSM steps.



```
[21]
✓ 11m
    #Keep the original images to ensure that the final adversarial image is within a defined distance from the original
    og_images = images.data

    #iteratively perform pgd process
    for i in range(iters):
      images.requires_grad = True
      outputs = model(images)
      model.zero_grad()
      cost = criterion(outputs, labels)
      cost.backward()

      #small FGSM step
      adv_images = images + step_size*images.grad.sign()
      eta = torch.clamp(adv_images - og_images, min=-eps, max=eps)
      images = torch.clamp(og_images + eta, min=0, max=1).detach_()

    return images

model.eval()

correct = 0
total = 0

# Iteratively go through each batch of images and labels in the test data loader
for images, labels in test_loader:
  images = images.to(device)
  labels = labels.to(device)

  #adverserial example generation using pgd method
  ae_images = pgd_attack(model, images, criterion, labels)

  outputs = model(ae_images)
  _, predicted = torch.max(outputs.data, 1)
  total += labels.size(0)
  correct += (predicted == labels).sum().item()
```

Figure 2, showing the pgd_attack function, which repeats small FGSM steps

4. Testing & Evaluation

4.1 Clean vs Adversarial Accuracy

Each attack was applied to all 10,000 test images with the model in evaluation mode, and the model's predictions on the adversarial images were compared against the original labels. Figure 3, below, compares clean and adversarial accuracy at an epsilon of 0.08.

The model classified 99.37 percent of the clean test images correctly, leaving 63 misclassified. FGSM lowered accuracy to 93.54 percent, so 646 images were misclassified. PGD lowered it further to 90.44 percent with 956 misclassified, which means that at the same epsilon the iterative method caused 376 more errors than the single step method.

```
for n, a in zip(names, accs):
  print('{:<15} {:.2f} %'.format(n, a))

plt.bar(names, accs)
plt.ylabel('Accuracy (%)')
plt.title('Clean vs adversarial accuracy')
plt.show()
```

```
Clean           99.37 %
FGSM (0.08)     93.54 %
PGD (0.08)      90.44 %
```

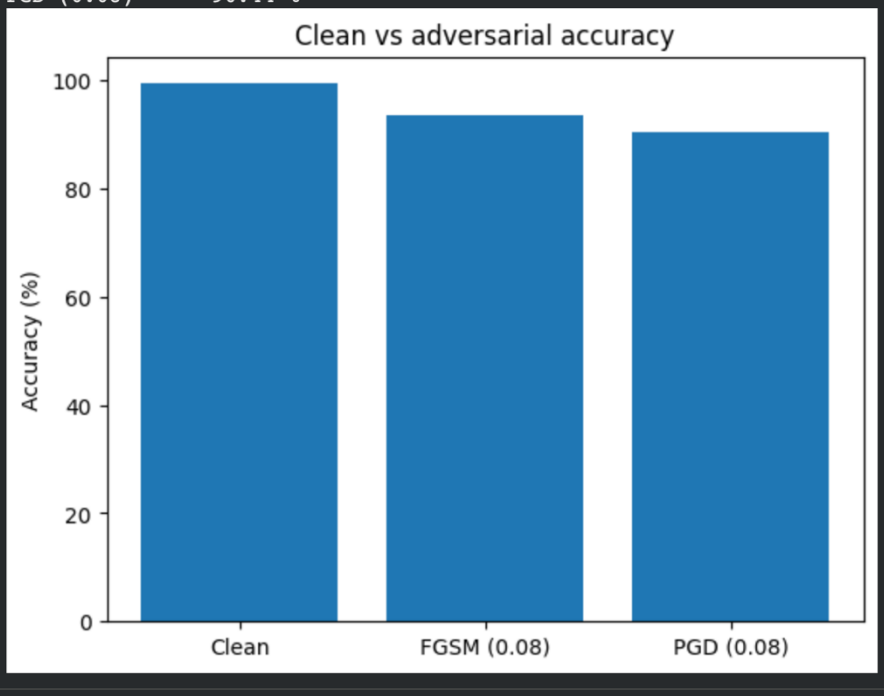


Figure 3, showing clean vs adversarial accuracy at ε = 0.08

4.2 Effects of Epsilon

Both attacks were repeated at epsilon values of 0, 0.05, 0.1, 0.15, 0.2, 0.25 and 0.3 on the full test set (see Figure 4). These experiments were run on a CPU runtime, because GPU access in Google Colab was limited without a Pro subscription. As a result, the PGD experiment took about an hour to complete, compared with about two minutes for FGSM, since PGD runs 40 forward and backward passes for every image.

Both curves start at the clean accuracy of 99.37 percent at an epsilon of 0, since no perturbation is added there. At 0.05 the two attacks barely differ, with FGSM at 96.88 percent and PGD at 95.66 percent, because most test images sit far enough from a decision boundary that a change this small cannot push them across. As epsilon grows, more images fall within reach of a boundary and accuracy begins to drop sharply.

For FGSM this steep drop happens between 0.1 and 0.2, where accuracy falls from 87.71 percent to 32.10 percent. It then levels off, reaching 15.82 percent at 0.25 and 9.38 percent at 0.3. PGD reaches its steep drop earlier, between 0.05 and 0.15, falling from 95.66 percent to 18.91 percent. It then reaches 0.52 percent at 0.2 and zero at 0.25, so at those values almost every test image can be pushed across a boundary.

The largest gap between the two methods is 45.39 points at an epsilon of 0.15, where FGSM still classifies 64.30 percent of images correctly against 18.91 percent for PGD. A larger epsilon makes both attacks stronger, but PGD consistently gets more out of the same epsilon, because it searches within that range step by step instead of

committing to a single gradient direction. PGD therefore needs a smaller and less visible change to cause the same damage as FGSM, a trade-off examined further in section 5.1.

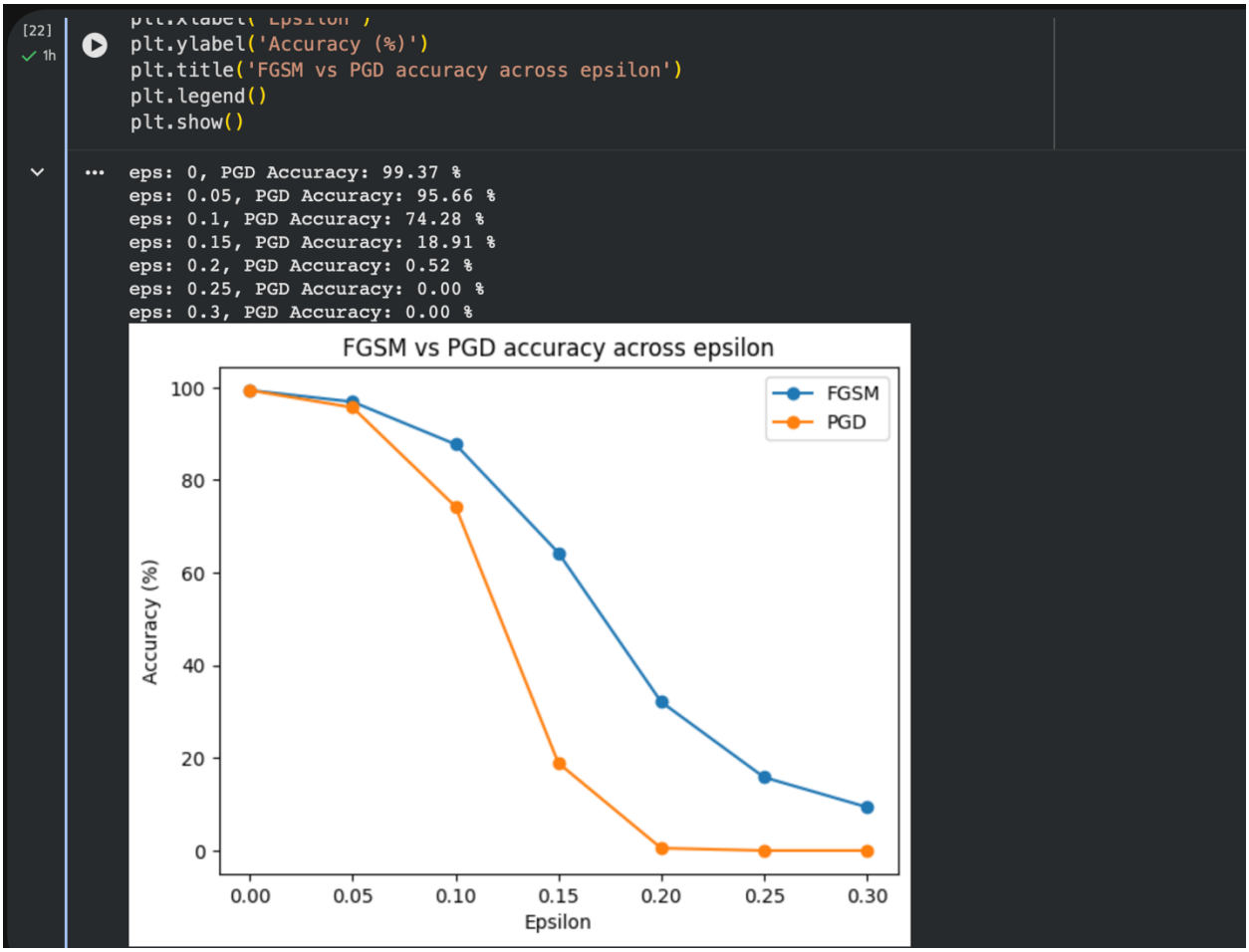


Figure 4, FGSM and PGD accuracy across epsilon

5. Visualization & Analysis

5.1 Perturbation and Perceptibility

In order to see how visible the perturbation becomes as epsilon grows, one test image, a handwritten 7, was attacked with FGSM at each epsilon value from 0 to 0.3, and each version was displayed with the model's prediction (see Figure 5 below).

The model kept predicting 7 through an epsilon of 0.15 and switched to 3 from 0.2 onward. At 0.05 the change is a faint gray haze over the black background, and the image would pass as clean. By 0.1 the background has turned into visible blocky texture. By 0.2 the stroke itself is dimmed and broken in places, which is also where the prediction flipped.

This shows the trade-off between perturbation magnitude and perceptibility. Small epsilon values are hard to see but fool the model on only a small share of images, while the values that defeat FGSM on most of the test set are clearly visible. MNIST makes this trade-off harsher than it would be on natural photographs, since its backgrounds are pure black and any brightening stands out against them. PGD shifts the trade-off in the attacker's favor because it causes the same damage at a smaller epsilon. It lowers accuracy to 74.28 percent at 0.1, a level FGSM only reaches somewhere between 0.1 and 0.15.

NB: Because of the long run times on the CPU runtime, this visual test was done for FGSM only.



Figure 5, one test digit under FGSM at each epsilon, labeled epsilon -> prediction

5.2 Original vs adversarial images

Five test images were displayed next to their FGSM versions at an epsilon of 0.15, along with the perturbation that separates them, computed as the adversarial image minus the original. The first two are shown in Figure 6, and all five can be viewed in the notebook on GitHub.



Figure 6, original (left), adversarial (middle) and perturbation (right) at epsilon = 0.15

All five digits (7, 2, 1, 0 and 4) kept their correct labels at this epsilon. That fits the 64.30 percent FGSM accuracy at 0.15, since most images still survive at this level. The perturbation column shows the digit as a dark shape on a

patchy white and gray background, which is a result of clipping. Stroke pixels are already close to 1, so they can only be pushed down, and the stroke is darkened by epsilon. Background pixels are at 0, so they can only be pushed up, and they appear as white patches wherever the gradient increased their intensity. FGSM therefore partly fades the digit while adding faint noise to the background around it.

5.3 Feature Maps for clean and adversarial inputs

The feature maps were produced the same way as in Project 1, by passing one test image, a handwritten 1, through the network up to each convolutional layer's activation. This was done once for the clean image and once for its FGSM version at an epsilon of 0.15. The first 8 channels of each layer are shown, with the clean maps on the top row and the adversarial maps on the bottom row. The Conv 1 maps are shown in Figure 7, and the Conv 2 and Conv 3 maps can be viewed in the notebook on GitHub.

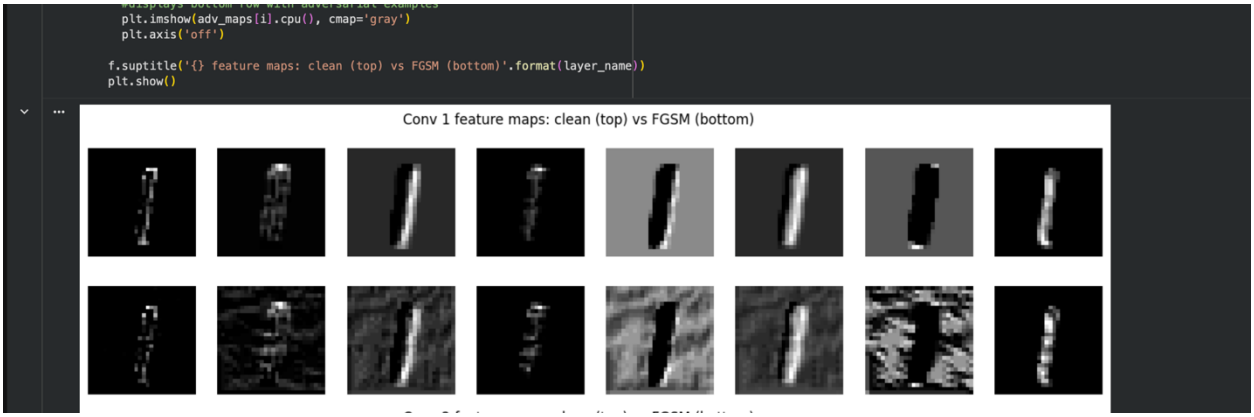


Figure 7, Conv 1 feature maps, clean (top) vs FGSM (bottom)

At the first layer, the channels that respond to the stroke still outline the 1 under attack. The channels that respond to contrast, however, pick up the background noise and fill with texture that is absent from the clean maps. At the second layer, several channels that were nearly blank on the clean image show scattered activations across the background. At the third layer the maps are small and sparse, and most keep their clean pattern, though some activations shift position.

The maps show the perturbation entering the first layer as background noise and spreading into channels that were inactive on the clean image as it moves deeper. The third layer is only 7 by 7, so the changes there look small. These are the features the classifier reads directly, though, so a few shifted activations at this layer are enough to change the prediction once epsilon is large enough.

5.4 Comparing FGSM and PGD

Both methods take their direction from the sign of the loss gradient, which is what lets such a small change per pixel fool the model. Each pixel moves by at most epsilon, but every change pushes the output toward the wrong class at the same time, so the effects add up across all 784 pixels of the image.

PGD was stronger than FGSM at every epsilon above zero. FGSM uses the full epsilon in one step and assumes the loss rises in a straight line along that step. This holds reasonably well for small values, so the two methods are only about a point apart at 0.05. At larger values the loss surface curves, and the single step overshoots or lands in a weaker spot. PGD instead re-reads the gradient at each of its 40 steps and keeps moving toward higher loss. The gap between the two methods grew until it peaked at 45.39 points at an epsilon of 0.15. It then narrowed as both attacks approached their lowest accuracy, so FGSM herefore overestimates how robust the model is.

The cost of that strength is compute. PGD runs 40 forward and backward passes per image where FGSM runs one. On the CPU runtime, the full range of epsilon values took about an hour for PGD against about two minutes for FGSM.

6. Hyperparameter Discussion

The attacks have three configured values: epsilon, the PGD step size and the number of PGD iterations. To explain the values chosen, each was tested separately. Epsilon was varied for both attacks in section 4.2. The step size and the number of iterations were varied for PGD at a fixed epsilon of 0.08.

Epsilon has the widest influence, since it sets how far any pixel is allowed to move and therefore both how damaging and how visible the attack is. For FGSM the steepest drop came between 0.1 and 0.2, where accuracy fell by 55.61 points. For PGD it came earlier, between 0.05 and 0.15, where accuracy fell by 76.75 points. An epsilon of 0.08 was used for the main comparison because it falls between 0.05, where the change in Figure 5 is barely visible, and 0.1, where the background texture becomes clear. At that value it still produces a measurable difference between the two methods.

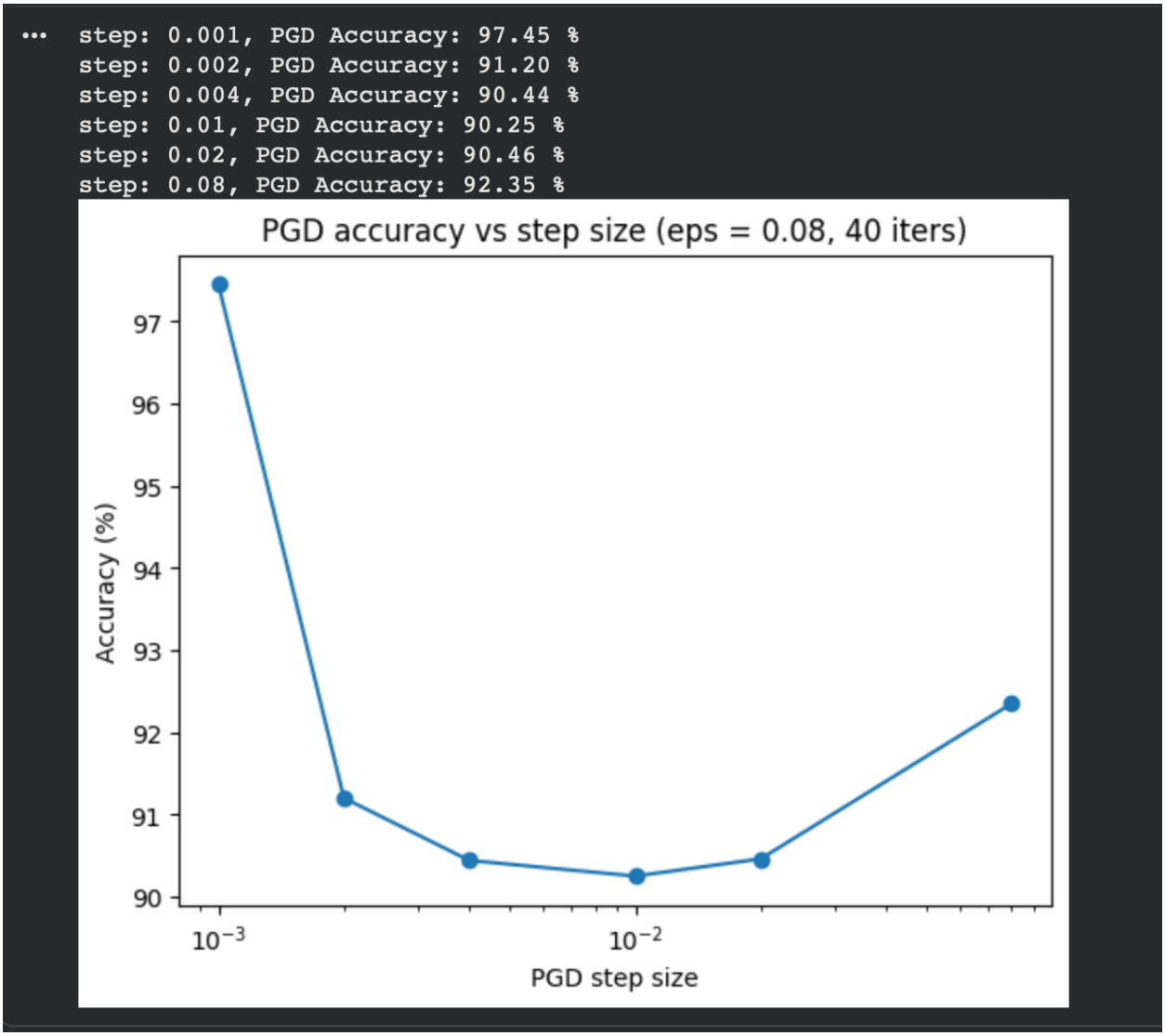


Figure 8, PGD accuracy vs step size at epsilon = 0.08 with 40 iterations

The step size controls how far each PGD iteration moves the image (see Figure 8). With 40 iterations, a step of 0.001 can move a pixel by at most 0.04 in total, only half of epsilon, so accuracy stayed at 97.45 percent. Once the step was large enough to reach the full epsilon within 40 iterations, from 0.002 onward, accuracy settled between 90.25 and 91.20 percent. Steps from 0.004 to 0.02 all gave results within 0.21 points of each other. At 0.08 each step

uses the whole epsilon at once, so PGD behaves like repeated FGSM and bounces around the edge of the allowed range instead of refining its direction. Accuracy rose back to 92.35 percent there. A step of 0.004 was chosen because it sits in the flat region where the attack is strongest, while still being small enough for 40 iterations to make careful adjustments.

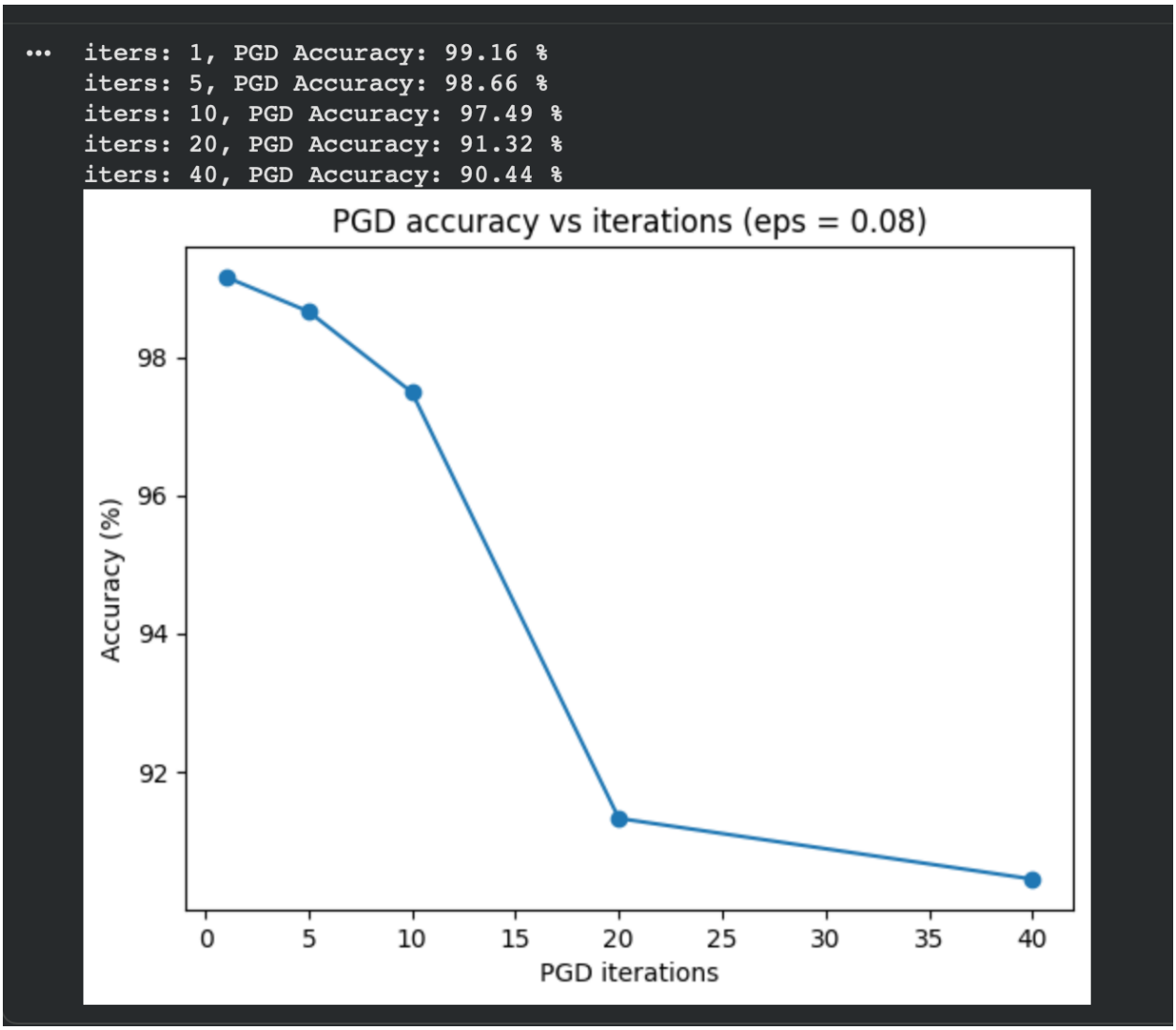


Figure 9, PGD accuracy vs number of iterations at epsilon = 0.08 with a step size of 0.004

The number of iterations showed the same pattern from the other side (see Figure 9 above). With a step size of 0.004, 1, 5 and 10 iterations can only move a pixel by 0.004, 0.02 and 0.04, so accuracy stayed between 97.49 and 99.16 percent. At 20 iterations the total possible change reaches the full epsilon of 0.08, and accuracy dropped to 91.32 percent. Doubling to 40 iterations lowered it by only another 0.88 points, to 90.44 percent. Forty iterations was chosen because the attack is close to full strength there. More iterations would add run time, which was already about an hour on the CPU runtime, for little further effect.

The two experiments show that what matters most is whether the step size and the number of iterations together let PGD reach the full epsilon. This is why, when epsilon was varied in section 4.2, the step size was set to one tenth of epsilon. That way 40 steps could always reach the full epsilon at every value tested.

The training configuration was carried over unchanged from Project 1: a learning rate of 0.001, a batch size of 256, ten epochs and a dropout rate of 0.5. Those values were discussed in that report. Keeping them fixed meant the model stayed the same throughout this lab, so any change in accuracy could be attributed to the attack settings alone.

Forty iterations were enough for PGD to drive accuracy to zero at the largest budgets, which suggests the attack was close to its full strength there. More iterations, or several random starting points inside the budget, could lower PGD accuracy slightly further at the middle budgets, at a proportional cost in run time.

7. Countermeasures and suggested improvements

Adversarial training is the most direct defense against these attacks. Each training batch would be mixed with PGD examples generated against the current model, so the network learns to classify correctly within epsilon of each image. PGD should be used to generate these examples rather than FGSM, since a model trained only against FGSM tends to resist that single step while staying open to iterative attacks. The cost is roughly 40 times the training compute per epoch, along with a small drop in clean accuracy.

Input preprocessing is a cheaper addition. MNIST digits are close to black and white, so binarizing each image or applying a small median filter before it reaches the model would remove much of the background noise shown in Figure 6. On the architecture side, a wider network gives adversarial training more capacity to learn the harder task. Any of these defenses should be tested against PGD rather than FGSM alone, since the 45.39-point gap at an epsilon of 0.15 shows how much a single step attack can underestimate a model's weakness.

The experiments themselves could also be improved by running them on the school's HPC cluster through Jupyter Notebook instead of a free Colab CPU runtime. With a GPU, the PGD experiments would drop from about an hour to a few minutes. That would leave time to repeat the perceptibility and feature map tests for PGD.

8. Conclusions

PGD was the stronger of the two attacks at every epsilon above zero. At an epsilon of 0.08, FGSM lowered accuracy from 99.37 percent to 93.54 percent and PGD lowered it to 90.44 percent. At 0.3, FGSM still left 9.38 percent while PGD left none. The two methods agree at small epsilon values but separate as epsilon grows, because PGD re-reads the gradient at every step while FGSM commits to one. The hyperparameter experiments showed that PGD reaches close to full strength once its step size and iterations allow it to cover the full epsilon.

The visualizations showed FGSM fading the digit's stroke and adding noise to the background. The feature maps showed that noise entering the first convolutional layer and spreading into channels that were inactive on the clean image. The main limitation is that only one model and white-box attacks were tested. The clearest next step is to adversarially train this network with PGD and repeat both attacks across the same range of epsilon values.

9. Appendix — Source Code

Link to GitHub repository

AI transparency - I used Claude as an assistant on this project, both for debugging and refining the notebook code and for drafting and revising the written sections of this report. The experimental results, the analysis and the conclusions are my own understanding of class material, and I reviewed and edited all generated code and text before including it.

End of Report